

STUDENT PERFORMANCE ANALYSIS

WHICH STUDENTS ARE THE TOP PERFORMERS?

```
SELECT  
STUDENT_ID, ROUND(OVERALL_SCORE) AS TOP_PERFORMERC  
FROM  
STUDENT  
ORDER BY 2 DESC  
LIMIT 10;
```

	student_id	top_performerc
▶	105745	97
	108453	95
	107374	94
	104667	93
	103598	93
	104526	92
	107472	92
	100012	92
	102135	92
	107667	92

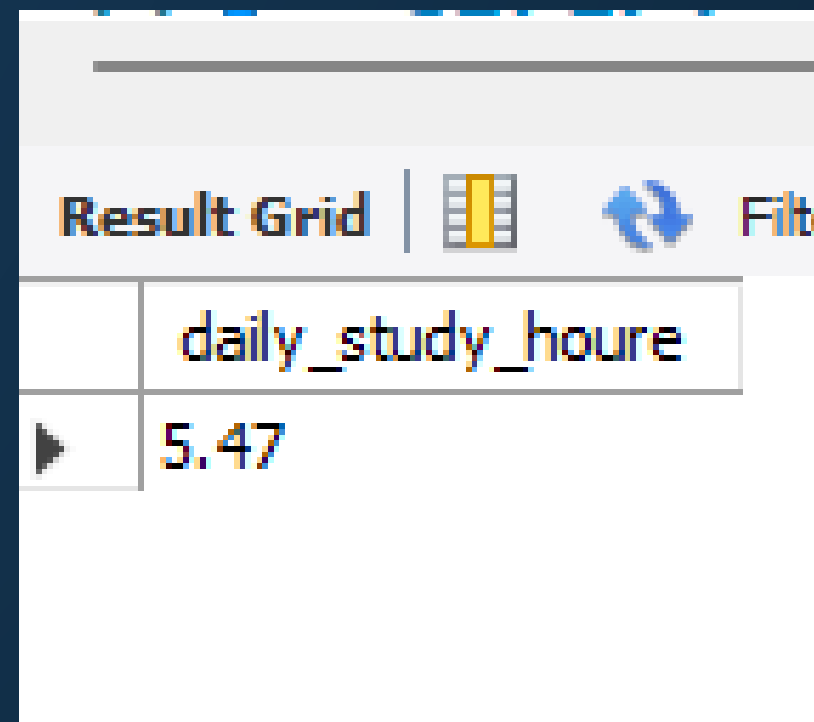
WHICH STUDENTS ARE AT RISK OF FAILING (OVERALL_SCORE < 50)?

```
SELECT
STUDENT_ID, ROUND(OVERALL_SCORE) AS RISK_OF_FAILING
FROM
STUDENT
WHERE
OVERALL_SCORE < 50
ORDER BY 2 DESC;
```

Result Grid			Filter Rows:	
	student_id	risk_of_failing		
▶	100428	50		
	100707	50		
	101025	50		
	101042	50		
	101318	50		
	101510	50		
	101531	50		
	101630	50		
	101631	50		
	101945	50		
	102227	50		
	102404	50		
	102435	50		

WHAT IS THE AVERAGE SCORE BASED ON DAILY STUDY HOURS?

```
SELECT  
ROUND(AVG(STUDY_HOURS_PER_DAY), 2) AS DAILY_STUDY_HOURE  
FROM  
STUDENT;
```



The screenshot shows a database query result grid. The header row is labeled 'daily_study_houre'. The first data row shows the value '5.47'. The grid has a toolbar with icons for 'Result Grid', a table icon, a refresh icon, and a 'Filter' button.

	daily_study_houre
▶	5.47

HOW DOES PERFORMANCE CHANGE BASED ON ATTENDANCE PERCENTAGE?

```
SELECT
    STUDENT_ID,
    ATTENDANCE_PERCENTAGE,
    ROUND(OVERALL_SCORE) AS PERFORMANCE
FROM
    STUDENT
ORDER BY 2 DESC , 3 DESC;
```

	student_id	attendance_percentage	performance
▶	100618	100	73
	103912	99.99	72
	106043	99.99	62
	106833	99.98	65
	109681	99.967	50
	100145	99.97	47
	104512	99.96	79
	109701	99.96	67
	101672	99.95	53
	103317	99.94	69
	108853	99.93	73
	104544	99.93	60
	104183	99.92	69

DOES INTERNET ACCESS IMPROVE STUDENT PERFORMANCE?

```
SELECT
  INTERNET_ACCESS,
  COUNT(*) AS STUDENT_COUNT,
  ROUND(AVG(OVERALL_SCORE), 2) AS AVE_PERFORMANCE,
  ROUND(MAX(OVERALL_SCORE), 2) AS MAX_PERFORMANCE,
  ROUND(MIN(OVERALL_SCORE), 2) AS MIN_PERFORMANCE
FROM
  STUDENT
GROUP BY 1;
```

	internet_access	student_count	ave_performance	max_performance	min_performance
*	Yes	5061	64.12	96.88	34.61
	No	4939	63.53	94.05	33.26

DO STUDENTS WHO TAKE EXTRA CLASSES PERFORM BETTER?

```
SELECT
  EXTRA_CLASSES,
  COUNT(*) AS STUDENT_COUNT,
  ROUND(AVG(OVERALL_SCORE), 2) AS AVE_PERFORMANCE,
  ROUND(MAX(OVERALL_SCORE), 2) AS MAX_PERFORMANCE,
  ROUND(MIN(OVERALL_SCORE), 2) AS MIN_PERFORMANCE
FROM
  STUDENT
GROUP BY 1
ORDER BY 1 DESC;
```

	extra_classes	student_count	ave_performance	max_performance	min_performance
►	Yes	5087	63.89	96.88	34.84
	No	4913	63.76	95.21	33.26



HOW DOES PARENT EDUCATION LEVEL AFFECT STUDENT PERFORMANCE?

```
SELECT
  PARENT_EDUCATION,
  COUNT(*) AS STUDENT_COUNT,
  ROUND(AVG(OVERALL_SCORE), 2) AS AVE_PERFORMANCE
FROM
  STUDENT
GROUP BY 1
ORDER BY 1 DESC;
```

Result Grid				Filter Rows:	Export:
	parent_education	student_count	ave_performance		
▶	PhD	2458	63.62		
	Master	2487	63.92		
	High School	2528	63.75		
	Bachelor	2527	64.01		

WHICH GRADE HAS THE HIGHEST NUMBER OF STUDENTS?

```
SELECT  
  GRADE, COUNT(STUDENT_ID) AS HIGHEST_STUDENT  
FROM  
  STUDENT  
GROUP BY 1  
ORDER BY 2 DESC  
LIMIT 1;  
SELECT * FROM STUDENT LIMIT 3;
```

Result Grid			Filter Rows:	
	grade	highest_student		
▶	C	5073		

COMPARE AVERAGE PERFORMANCE BY GENDER

```
SELECT
  GENDER,
  ROUND(AVG(FINAL_EXAM_SCORE), 2) AS
    AVG_FINAL_EXAM,
  ROUND(AVG(MIDTERM_SCORE), 2) AS AVG_MIDTERM,
  ROUND(AVG(OVERALL_SCORE), 2) AS AVG_OVERALL
FROM
  STUDENT
GROUP BY GENDER
ORDER BY AVG_OVERALL DESC;
```

Result Grid

 Filter Rows:

Export: 

	gender	avg_final_exam	avg_midterm	avg_overall
▶	Female	65	62.16	63.83
	Male	64.96	62.48	63.82

FIND TOP 3 STUDENTS FROM EACH GENDER

```
WITH TOP_STUDENT AS (  
  SELECT GENDER,STUDENT_ID,ROUND(OVERALL_SCORE)  
        AS SCORE,RANK()  
        OVER(PARTITION BY GENDER ORDER BY  
              OVERALL_SCORE DESC) AS  
              RANKING FROM STUDENT  
        )  
SELECT * FROM TOP_STUDENT  
WHERE RANKING<=3;
```

Result Grid		Filter Rows:		
	gender	student_id	score	ranking
▶	Female	107374	94	1
	Female	103598	93	2
	Female	104667	93	3
	Male	105745	97	1
	Male	108453	95	2
	Male	100012	92	3

WHICH COMPONENT (ASSIGNMENT, MIDTERM, FINAL) IMPACTS PERFORMANCE THE MOST?

```
SELECT  
  'ASSIGNMENT' AS COMPONENT,  
  CORR(ASSIGNMENT_SCORE, OVERALL_SCORE) AS CORRELATION  
FROM STUDENT
```

UNION ALL

```
SELECT  
  'MIDTERM',  
  CORR(MIDTERM_SCORE, OVERALL_SCORE)  
FROM STUDENT
```

UNION ALL

```
SELECT  
  'FINAL',  
  CORR(FINAL_EXAM_SCORE, OVERALL_SCORE)  
FROM STUDENT;
```

FIND STUDENTS WITH LOW ATTENDANCE BUT HIGH SCORES

```
SELECT
STUDENT_ID, ATTENDANCE_PERCENTAGE, OVERALL_SCORE
FROM
STUDENT
WHERE
ATTENDANCE_PERCENTAGE < (SELECT
AVG(ATTENDANCE_PERCENTAGE)
FROM
STUDENT)
AND OVERALL_SCORE > (SELECT
AVG(OVERALL_SCORE)
FROM
STUDENT);
```

	student_id	attendance_percentage	overall_score
▶	100003	45	66.411
	100004	51.13	65.600500000000001
	100006	55.47	65.489
	100010	44.05	77.053
	100020	56.34	64.84
	100029	61.98	81.302000000000002
	100037	68.31	65.9655
	100039	64.89	70.596
	100041	41.21	67.08
	100047	66.72	64.7235

70.596

DETECT ANOMALIES WHERE STUDY HOURS ARE HIGH BUT SCORES ARE LOW

```
SELECT
STUDENT_ID, STUDY_HOURS_PER_DAY, OVERALL_SCORE
FROM
STUDENT
WHERE
STUDY_HOURS_PER_DAY > (SELECT
AVG(STUDY_HOURS_PER_DAY)
FROM
STUDENT)
AND OVERALL_SCORE < (SELECT
AVG(OVERALL_SCORE)
FROM
STUDENT);
```

	student_id	study_hours_per_day	overall_score
▶	100011	7.36	62.9825
	100013	6.72	49.066
	100018	6.04	61.067499999999995
	100022	9.58	50.800000000000004
	100023	7.47	58.280499999999996
	100024	9.37	52.688500000000005
	100028	7.53	58.0055
	100032	6.08	58.3225
	100036	6.55	44.147500000000001
	100044	6.61	61.121999999999999

DOES PARTICIPATION SCORE IMPACT OVERALL PERFORMANCE?

```
SELECT
PARTICIPATION_SCORE, AVG(OVERALL_SCORE) AS AVG_OVERALL
FROM
STUDENT
GROUP BY PARTICIPATION_SCORE
ORDER BY PARTICIPATION_SCORE;
```

Result Grid			Filter Rows:
	participation_score	avg_overall	
▶	10	56.65375	
	10.02	53.921	
	10.03	71.7075	
⋮	10.06	43.152	
	10.09	55.471999999999994	
	10.11	50.715	
	10.12	67.318	
	10.14	51.103500000000004	
	10.15	57.8695	

WHICH STUDENTS ARE THE MOST CONSISTENT PERFORMERS
(LOW VARIANCE)?

```
SELECT
  STUDENT_ID,
  ROUND((ASSIGNMENT_SCORE + MIDTERM_SCORE + FINAL_EXAM_SCORE) / 3) AS
    AVERAGE_SCORE,
  ROUND(POW(ASSIGNMENT_SCORE - (ASSIGNMENT_SCORE + MIDTERM_SCORE +
    FINAL_EXAM_SCORE) / 3,
    2) + POW(MIDTERM_SCORE - (ASSIGNMENT_SCORE + MIDTERM_SCORE +
    FINAL_EXAM_SCORE) / 3,
    2) + POW(FINAL_EXAM_SCORE - (ASSIGNMENT_SCORE + MIDTERM_SCORE +
    FINAL_EXAM_SCORE) / 3,
    2)) AS VARIANCE
  FROM
    STUDENT
  ORDER BY VARIANCE
  LIMIT 10;
```

	student_id	average_score	variance
▶	106824	85	0
	109789	32	0
	101757	84 84	1
	100399	93	1
	105490	42	1
	105069	98	1
	106648	74	1
	107851	46	1
	103331	92	1

FIND AVERAGE STUDY HOURS PER GRADE

```
SELECT
    GRADE,
    ROUND(AVG(STUDY_HOURS_PER_DAY), 2) AS AVERAGE_STUDY_HOURE
FROM
    STUDENT
GROUP BY 1;
```

	grade	average_study_houre
▶	D	5.51
	B	5.51
	C	5.44
	A	5.29
	F	5.38

IDENTIFY STUDENTS WITH HIGH ATTENDANCE BUT LOW SCORES

```
SELECT
STUDENT_ID, ATTENDANCE_PERCENTAGE, OVERALL_SCORE
FROM
STUDENT
WHERE
ATTENDANCE_PERCENTAGE > (SELECT
AVG(ATTENDANCE_PERCENTAGE)
FROM
STUDENT)
AND OVERALL_SCORE < (SELECT
AVG(OVERALL_SCORE)
FROM
STUDENT);
```

	student_id	attendance_percentage	overall_score
▶	100001	84.8	53.948499999999996
	100014	92.04	48.274499999999996
	100016	70.69	62.959
	100017	82.4	55.2335
	100021	89.92	50.682
	100023	86.39	58.280499999999996
	100035	88.44	63.649499999999996
	100044	83.02	61.121999999999999
	100045	74.96	62.0135

SEGMENT STUDENTS INTO HIGH, MEDIUM, LOW PERFORMERS

```
SELECT
  STUDENT_ID,
  GRADE,
  CASE
    WHEN OVERALL_SCORE > 80 THEN 'HIGH'
    WHEN OVERALL_SCORE BETWEEN 70 AND 80 THEN 'MEDIUM'
    ELSE 'LOW'
  END AS CATEGORY
FROM
  STUDENT;
```

	student_id	grade	category
▶	100000	D	Low
	100001	D	Low
	100002	B	High
	100003	C	Low
	100004	C	Low
	100005	C	Low
	100006	C	Low
	100007	B	Medium
	100008	D	Low
	100009	C	Low

WHO ARE THE TOP 10% STUDENTS?

```
SELECT * FROM  
(SELECT STUDENT_ID,OVERALL_SCORE ,  
NTILE(10)OVER(ORDER BY OVERALL_SCORE DESC) AS RANKING  
FROM STUDENT) AS T  
WHERE RANKING=1;
```

	student_id	overall_score	ranking
▶	105745	96.879500000000001	1
	108453	95.2065	1
	107374	94.047	1
	103598	93.415000000000002	1
	104667	92.7165	1
	100012	92.383000000000001	1
	107472	91.934499999999999	1
	107667	91.864499999999999	1
	108563	91.702	1
	102135	91.682999999999999	1

Result 29 ✕

WHICH STUDENTS IMPROVED FROM MIDTERM TO FINAL EXAM?

```
SELECT
    STUDENT_ID,
    MIDTERM_SCORE,
    FINAL_EXAM_SCORE,
    ROUND((FINAL_EXAM_SCORE - MIDTERM_SCORE)) AS IMPROVED_STUDENT
FROM
    STUDENT
ORDER BY 4 DESC;
```

student_id	midterm_score	final_exam_score	improved_student
102887	25.95	99.88	74
106602	25.84	99.87	74
103389	25.98	99.5	74
105996	25.52	99.33	74
102424	26.36	99.2	73
104760	25.58	98.78	73
105620	26.31	98.96	73
109123	26.71	99.69	73
106060	26.01	99.07	73
104844	26.33	98.72	72

CREATE A WEAK STUDENT LIST FOR RISK PREDICTION MODEL

```
SELECT
    STUDENT_ID,
    ATTENDANCE_PERCENTAGE,
    STUDY_HOURS_PER_DAY,
    ROUND(OVERALL_SCORE) AS OVERALL_SCORE
FROM STUDENT
WHERE
    ATTENDANCE_PERCENTAGE < 60
    AND OVERALL_SCORE < (
        SELECT AVG(OVERALL_SCORE) FROM STUDENT
    )
    AND STUDY_HOURS_PER_DAY < 2;
```

student_id	attendance_percentage	study_hours_per_day	overall_score
100005	53.16	1.79	59
100073	41.68	1.77	54
100090	53.56	1.33	63
100137	56.42	1.72	63
100225	42.86	1.52	60
100342	56.36	1.82	57
100525	47.83	1.31	57
100565	54	1.22	61
100604	55.88	1.36	61
100661	46.58	1.95	62